

ADT Specifications

CSC322 Group E • Design Portfolio, Part 2 of 3 Companion documents: [grammar.md](#), [design-portfolio.md](#)

Every abstract data type in `CalcEngine.Core` is specified here with an **abstraction function** (what a representation *means*), a **representation invariant** (which representations are legal), and the reasoning behind the representation chosen. The same text appears as XML documentation on each type, so it is visible in IntelliSense and cannot drift away from the code without someone noticing.

Notation: `AF(x)` is the abstract value denoted by concrete value `x`; `RI(x)` is the predicate every reachable `x` must satisfy.

1. Why these are the ADTs

The brief says the expression tree and the dependency graph are the heart of the project. They are, but neither is expressible without three smaller types that carry the invariants everything else assumes: an address that is always on the grid, a range that is always normalised, and a value that always knows what it is. Getting those three right is what makes the two big ones simple.

§	Type	Kind	Immutable
2	<code>CellAddress</code>	value	yes
3	<code>CellRange</code>	value	yes
4	<code>CellValue</code>	tagged union	yes
4.1	<code>ErrorValue</code>	value	yes
5	<code>Expression</code> (+ 9 node types)	composite tree	yes
6	<code>SourceSpan</code> , <code>CellReference</code>	value	yes
7	<code>DependencyGraph</code>	mutable structure	no
7.1	<code>RangeDependencyIndex</code>	mutable structure	no
8	<code>Workbook</code> / <code>Sheet</code>	mutable aggregate	no
9	<code>CommandStack</code>	mutable structure	no
10	<code>DuplicateReport</code> , <code>FindMatch</code>	value	yes

2. `CellAddress`

The location of one cell inside one sheet, in A1 notation.

Representation. Two `int` fields, `_columnIndex` and `_rowIndex`, both **zero-based**; a `readonly struct`, 8 bytes.

AF. $AF(c)$ = the cell at column $c._columnIndex + 1$, row $c._rowIndex + 1$ of *some* sheet. The sheet is deliberately not part of the value: pairing an address with a sheet is `SheetCellAddress`'s job (§2.1), and keeping them apart means a formula copied between sheets does not carry a stale sheet name.

RI. $0 \leq _columnIndex < 16384 \wedge 0 \leq _rowIndex < 1048576$.

Why zero-based storage. So that `default(CellAddress)` — the value C# hands to an uninitialised array element or a `default` expression — is `A1` and satisfies the invariant. Had the fields been the 1-based numbers, the default value would be the off-grid `(0, 0)`, and every method would need to defend against a value the type system says exists. This costs one addition per property read and removes an entire class of bug. (It did produce one, in the GUI: `default(CellAddress) == A1` made a "has the selection moved?" test wrong on the first render. That is recorded in the AI collaboration log, entry 14.)

Operations and their contracts.

Operation	Contract
<code>new CellAddress(col, row)</code>	1-based; throws <code>ArgumentOutOfRangeException</code> off the grid. The only way to break the RI is not to.
<code>TryParse(text, out a)</code>	Total. Accepts and discards <code>\$</code> markers: absoluteness belongs to a <i>reference</i> , not to a location.
<code>Parse(text)</code>	Partial; throws <code>FormatException</code> .
<code>ToA1()</code>	$\forall a. \text{Parse}(a.ToA1()) = a$ — a proven round trip (<code>ToA1_IsTheInverseOfParse</code>).
<code>ColumnName(n)</code> / <code>TryParseColumnName</code>	Bijjective base-26; mutual inverses on <code>1...16384</code> .
<code>Offset(dc, dr)</code>	Partial; throws off the grid.
<code>TryOffset(dc, dr, out a)</code>	Total. Callers wanting <code>#REF!</code> use this one.
<code>CompareTo</code>	Reading order: row-major, then column.

Equality is structural, so an address is usable as a dictionary key — which the sheet index and the range dependency index both rely on.

2.1 SheetCellAddress

AF. $AF(s)$ = the cell $AF(s.Address)$ of the sheet named $s.Sheet$. **RI.** $s.Sheet \neq null \wedge s.Sheet \neq ""$. Names compare case-insensitively, as in Excel, so `Marks!B2` and `marks!B2` denote the same cell.

3. CellRange

A rectangular block of cells, closed on both corners.

Representation. Two `CellAddress` fields, `TopLeft` and `BottomRight`.

AF. $AF(r) = \{ (col, row) \mid r.TopLeft.Column \leq col \leq r.BottomRight.Column \wedge r.TopLeft.Row \leq row \leq r.BottomRight.Row \}$ — the set of cells in the closed rectangle. A single cell is the range whose corners coincide.

RI. $TopLeft.Column \leq BottomRight.Column \wedge TopLeft.Row \leq BottomRight.Row$.

How the invariant is kept. The constructor takes *two corners* and normalises them, rather than taking a top-left and a bottom-right and trusting the caller. A selection dragged upwards produces the same range as the same selection dragged downwards, and `D9:B2` in a formula is the same range as `B2:D9`. This is why `RangeReferenceExpression` keeps `From` and `To` separately from `Range`: the *reference* must print back as the user wrote it, while the *range* is normalised for computation.

Laziness. `Cells()` is an iterator and `CellCount` is arithmetic (and `long`, since the whole grid is 17,179,869,184 cells and overflows `int`). A range covering the entire grid is therefore a cheap value, which matters because `RangeDependencyIndex` must be able to reason about one without materialising it.

4. `CellValue`

The value of a cell: blank, number, text, boolean or error.

Representation. A `readonly struct` of one `double _number`, one `object? _payload` and a `CellValueKind Kind`: 24 bytes, no allocation for numbers.

AF.

```
AF(v) = blank           when v.Kind = Blank
      = v._number       when v.Kind = Number
      = (string)v._payload when v.Kind = Text
      = (v._number != 0)  when v.Kind = Boolean
      = (ErrorValue)v._payload when v.Kind = Error
```

RI.

```
Kind = Text    => _payload is string
Kind = Error   => _payload is ErrorValue
Kind ∈ {Blank, Number, Boolean} => _payload = null
Kind = Boolean => _number ∈ {0, 1}
Kind = Number  => IsFinite(_number)
```

Why the last clause matters, and how it is enforced. A spreadsheet has no representation for NaN or infinity; Excel reports `#NUM!` and `#DIV/0!`. If a `CellValue` could hold `double.NaN`, every consumer would have to check, and one that forgot would put `NaN` in a student's transcript. So *nothing in the engine constructs a numeric `CellValue` from a computed double directly*: every arithmetic path

goes through `ValueCoercion.FromDouble`, which maps the two non-finite cases onto errors. `CellValue.Number` remains public for literals, where finiteness is already known.

Reading the wrong variant throws. `CellValue.Text("x").AsNumber` raises `InvalidOperationException`. That is not the same thing as a type error in a formula: `"x"+1` returns `#VALUE!`, a value the user can see. The exception is reserved for a bug in the *caller*, and the distinction is what keeps "spreadsheet errors are values" honest.

Equality is structural within a kind, and text is compared **ordinally** — case-sensitively — even though the `=` operator in a formula is case-insensitive. Value identity and formula equality are different questions: a change-notification path that treated `"PASS"` and `"pass"` as the same value would fail to tell the grid to repaint.

4.1 `ErrorValue`

AF. `AF(e)` = the error condition `e.Kind`, displayed as `e.Code`. `e.Detail` is diagnostic prose for the client — a cycle path, an unknown function name — and is **not** part of the abstract value.

RI. `Code` is the canonical spelling of `Kind`, non-empty; the instance is immutable; `ErrorValue.All` is indexed by `ErrorKind`.

Equality is by `Kind` alone. Two `#REF!` cells hold the same value even if one explains itself more fully. Without this, attaching a message to an error would register as a value change and the engine would notify observers of changes that did not happen — and, worse, a cycle's error message differing per cell would make every recalculation report every cycle cell as changed.

The eight conditions are `#DIV/0!`, `#VALUE!`, `#REF!`, `#NAME?`, `#NUM!`, `#N/A`, `#CIRC!` and `#PARSE!`. The last two are ours: Excel handles circularity with a dialogue box and refuses malformed formulas at entry, neither of which an API can do.

5. `Expression` — the expression tree

A Composite of literals, references, operators and function calls that denotes a function from an evaluation context to a `CellValue`.

Representation. An abstract base with nine sealed subclasses: `NumberLiteralExpression`, `TextLiteralExpression`, `BooleanLiteralExpression`, `ErrorLiteralExpression`, `CellReferenceExpression`, `RangeReferenceExpression`, `UnaryOperatorExpression`, `BinaryOperatorExpression`, `FunctionCallExpression`.

AF. `AF(e)` = the function `context ↦ CellValue` defined by structural recursion:

```

AF(NumberLiteral n)      (ctx) = n.Value
AF(TextLiteral t)        (ctx) = t.Value
AF(CellReference r)      (ctx) = ctx.GetCellValue(r.SheetName, r.Address)
AF(RangeReference r)     (ctx) = #VALUE!           (a range has no scalar value)
AF(Unary(op, x))         (ctx) = op(AF(x)(ctx))
AF(Binary(op, l, r))     (ctx) = op(AF(l)(ctx), AF(r)(ctx))
AF(FunctionCall(f, args)) (ctx) = ctx.Resolve(f)(args, ctx)    -- args unevaluated

```

RI. * The tree is finite and acyclic. * Every child reference is non-null. * A `FunctionCallExpression` has a non-empty, upper-cased `Name`. * The arity of an operator node matches its operator (structural, by class). * `TextLiteralExpression.Value` is *unescaped*: doubled quotes have already been collapsed.

Acyclicity is free. Children are assigned in the constructor and the classes are immutable, so no node can become its own descendant. There is no `Parent` pointer and no mutation, which is also what makes it safe for the undo history to hold a reference to an old tree without copying it.

Two traversal mechanisms, deliberately.

- `CellValue Evaluate(IEvaluationContext)` — a virtual method on each node: the **Interpreter** pattern. Evaluation is the hot path (a full recalculation walks every tree in the workbook), and one virtual call beats a visitor's double dispatch.
- `TResult Accept<TResult>(IExpressionVisitor<TResult>)` — the **Visitor** pattern, for the open-ended traversals: `FormulaPrinter`, `ReferenceCollector`, `TextLiteralFinder`. A new traversal costs a new class rather than a new method on nine node types.

Choosing one mechanism for everything would have been tidier and worse: an Interpreter-only design puts printing and dependency extraction on the node classes, and a Visitor-only design pays double dispatch on every cell of every recalculation.

`FormulaPrinter` is specified as an inverse. For every `e` produced by the parser, `Parse(Print(e))` yields a tree structurally equal to `e`; brackets appear only where removing them would change the tree. That property is what makes it safe for Find & Replace to write a rewritten formula back, and it is tested by round-tripping (`Print_IsAnInverseOfParse`) rather than asserted.

6. `SourceSpan` and `CellReference`

`SourceSpan`. `AF(s)` = the characters `[s.Start, s.Start + s.Length)` of the formula the node was parsed from. **RI:** `Start ≥ 0 ∧ Length ≥ 0`. It is **metadata**: not part of the abstract value of an expression, and takes no part in equality. It exists so that a rewrite can splice new text into the user's original spelling — preserving their spacing — instead of re-printing the whole formula.

`CellReference`. `AF(r)` = the cell `AF(r.Address)`, reached by a reference whose column is absolute iff `r.ColumnAbsolute` and whose row is absolute iff `r.RowAbsolute`. Absoluteness does not change which cell is read, so it takes no part in the dependency graph; it matters only when a formula is copied or printed.

7. DependencyGraph

Who reads whom, and in what order they must be recomputed.

Representation. Cells are dense `int` identifiers. Three parallel arrays indexed by identifier — `_precedents`, `_dependents`, `_rangePrecedents` — plus a `RangeDependencyIndex`, plus reusable traversal scratch (`_visitStamp`, `_state`, `_frameIndex`, `_childBuffer`, `_frames`).

AF. `AF(g)` = a directed graph whose vertices are cell identifiers and which contains the edge `u → v` exactly when the formula stored in `v` reads `u`, either directly or through a range containing `u`. The direction is "flows into", so a topological order of `AF(g)` is a safe evaluation order.

RI.

```

∀u, v. v ∈ _precedents[u] ⇔ u ∈ _dependents[v]      (the direct edges agree)
∀v.      every range in _rangePrecedents[v] is filed in _rangeIndex against v,
          and appears nowhere else in the index
          no adjacency list contains a duplicate
          all arrays have length ≥ the largest identifier + 1

```

The first clause is the one that bites: it is why `ClearPrecedents` must walk the old precedents and remove the back-edge from each, and why editing a formula is not simply "overwrite the precedent list".

Why integer identifiers. Adjacency becomes array indexing rather than hashing, one graph spans every sheet, and — decisively — the traversal marks become flat arrays that can be stamped instead of cleared.

Why generation stamps. A visited-set that is cleared costs `O(V)` per traversal. In a 100,000-cell workbook, clearing 100,000 flags on every keystroke costs more than the entire propagation it is meant to support. Instead `_currentStamp` is incremented and a vertex counts as unvisited when `_visitStamp[v] ≠ _currentStamp`. The measured effect is in `benchmarks.md` §4: a single edit through 500 dependents takes 0.2 ms against a 50 ms budget.

`TopologicalOrderFrom(seeds, ...)`. Depth-first search over the "flows into" edges, pushing each vertex after its descendants and reversing at the end — the classical topological sort, restricted to the subgraph reachable from `seeds` so that cost follows what changed rather than the size of the workbook.

- *Explicitly stacked, not recursive.* A chain of 20,000 dependent cells is an ordinary import artefact; a recursive walk meets it with a `StackOverflowException`, which cannot be caught, so the client simply loses the process. `ALongCycleDoesNotOverflowTheStack` and `ADeepChainDoesNotOverflowTheStack` hold this line at 20,000.
- *A back edge to a vertex still on the stack is a cycle.* It is recorded and then **not followed**, so the traversal still terminates and still orders everything outside the cycle correctly. The caller marks the ring's members `#CIRC!` instead of evaluating them, and cells downstream of the ring receive the error by ordinary error propagation.

- *The reported path is canonical.* The stack holds the ring in "is read by" order because that is the direction of travel; a user thinks in the other direction, so the tail is reversed, and the result is then rotated to start at the smallest address. The same broken workbook therefore always yields the same sentence — `A1 → B3 → C7 → A1` — no matter which cell the user typed last, which `TheCycleIsReportedTheSameWayWhicheverCellClosedIt` checks.

7.1 RangeDependencyIndex

AF. `AF(index)` = the set of range dependencies `{(sheet, range, dependent)}` it holds, irrespective of how they are filed.

RI. Every dependency appears **either** in `_large` **or** under exactly the blocks its range intersects, never both; no block list is empty.

The design problem. `=SUM(B2:B45)` depends on 44 cells. Expanding that into 44 edges is affordable once and ruinous at scale — but the fatal objection is not cost. It is that **typing into a cell that was empty when the formula was written must still trigger the formula**, and an empty cell has no edges to expand into.

The design. The grid is cut into 64×64 blocks. A range is filed under every block it touches; a changed cell is looked up by its own block and the few candidate ranges filed there are tested for containment. A range spanning more than 1,024 blocks goes into a small always-scanned list instead, so a pathological `A1:XFD1048576` costs one list entry rather than four million index entries. `WritingIntoAPreviouslyEmptyCellOfARangeStillTriggersIt` is the test that pins the property the whole structure exists for.

8. Workbook and Sheet

Workbook AF. `AF(w)` = a mapping from every `SheetCellAddress` to a pair (content, value), together with an undo history.

Workbook RI.

```
sheet names are unique, case-insensitively
every identifier belongs to exactly one sheet's address map, and
    _cells[id] records the sheet and address that map back to it
the dependency graph holds  $u \rightarrow v$  exactly when  $v$ 's stored tree reads  $u$ ,
    and holds no edges for cells whose content is a literal or failed to parse
CONSISTENCY: after any public call returns with automatic calculation on,
    every formula cell not in a cycle holds the value obtained by evaluating
    its tree against the current values of the cells it reads
```

The consistency clause is the engine's whole promise. It is temporarily suspended inside `ApplyEdits` — between storing new content and finishing the recalculation — and restored before the method returns. With `AutomaticCalculation` off it is suspended until `CalculateNow`, which is the entire meaning of manual mode.

Sheet AF. `AF(s)` = a partial function from `CellAddress` to content, plus a name; addresses outside the domain are blank.

Sheet RI. Every identifier in `IdByAddress` addresses a cell record *of this sheet*. The map holds an entry for every cell that has been **written to or merely referred to** — a formula reading an empty cell needs that cell to have an identity in the graph — which is why `PopulatedCells()` filters on content rather than trusting the map.

Why cells live in one flat array. A `CellRecord[]` indexed by identifier, not per-sheet dictionaries of objects: array-speed adjacency, one graph across sheets, and a 100,000-cell workbook costs a handful of allocations instead of 100,000. Each record holds `Content`, the parsed `Formula`, the `Literal` value and the last computed `Value`. `Literal` and `Value` are separate on purpose: without it, applying new content would overwrite the old value before the recalculation could report what it used to be, and every `CellChange` for an edited literal would claim the value had not changed.

9. `CommandStack`

AF. `AF(h)` = the sequence of operations performed and not yet undone (`_undo`, oldest first), together with the sequence undone and not yet redone (`_redo`, most recently undone last).

RI.

```
0 ≤ _undo.Count ≤ Limit
every command in either stack has been executed at least once,
    so its captured "previous content" is valid
_redo is empty immediately after any call to Execute
```

The limit counts operations, not cells. Replacing 4,000 occurrences of a course code is one operation and one press of Ctrl+Z. The brief asks for 100; the default is 100 and it is configurable.

Commands record what they did, not a snapshot. A hundred-deep history of a 100,000-cell workbook costs a hundred small objects rather than ten million cells. `SetCellsCommand` captures the previous content of exactly the cells it writes, and it re-captures on every execution rather than only the first, so a redo after intervening undos still restores the right content.

Undo re-enters the ordinary recalculation path. That is why undoing a formula edit restores its *dependency edges* and not merely the text that was showing — `UndoOfAFormulaEditRestoresItsDependencies` changes the old precedent afterwards and requires the cell to follow it.

10. Feature value types

`FindMatch`. `AF(m)` = the occurrence of the search text at `[m.Start, m.Start + m.Length)` within `m.SearchedText`, which is the content or the displayed value of `m.Address` according to `m.Target`. A reference type, so that "no match" is a null reference and `FindNext(...)?.Address` reads naturally.

`DuplicateReport`. `AF(r)` = the partition of the scanned records into equality groups, restricted to groups of size > 1 , plus the count of distinct records. `RI` : every group has ≥ 2 occurrences; occurrences within a group are in reading order, so `First` is genuinely the one a removal keeps; `_duplicateCells` contains exactly the cells of the non-first occurrences, which is what a grid shades.

The record key is a string, built with a one-character type tag and a length prefix per field. The tag keeps the number `5` apart from the text `"5"` — an imported column of text marks must not appear to duplicate a typed one. The length prefix keeps the record `{"a", "b"}` apart from `{"ab", ""}`, which a naive concatenation would declare identical.